Hands-on Activity 3.1 Linked Lists

OUTPUT

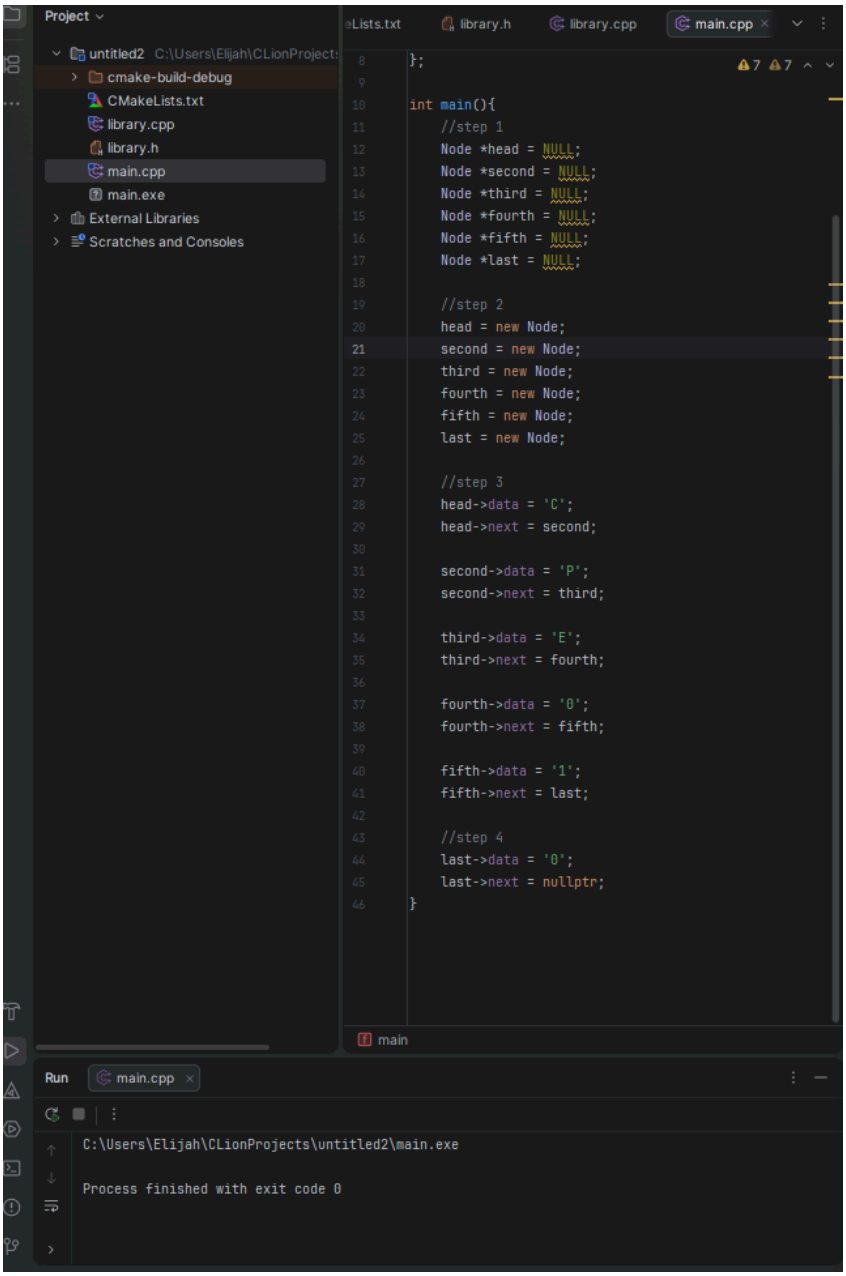
screenshot	

Table 3-1. Output of Initial/Simple Implementation

Traversal	void displayList() {
	  Node *temp = head;
	  if (temp == NULL) {
	  cout << "List is empty." << endl;
	  return;
	  }
	  cout << "List: ";
	  while (temp != NULL) {
	  std::cout << temp->data;
	  if (temp->next != NULL) std::cout << " -> ";
	  temp = temp->next;
	  }
	  std::cout << std::endl;
	}

Insertion at head	void insertAtHead(char value) {
	  Node *newNode = new Node;
	  newNode->data = value;
	  newNode->next = head;
	  head = newNode;
	}

Insertion at any part of the list	void insertAfter(Node *prevNode, char value) {
	  if (prevNode == NULL) {
	  std::cout << "Previous node cannot be null." << endl;
	  return;
	  }
	  Node *newNode = new Node;
	  newNode->data = value;
	  newNode->next = prevNode->next;
	  prevNode->next = newNode;
	}

Insertion at the end	void insertAtEnd(char value) {
	  Node *newNode = new Node;
	  Node *last = head;
	  newNode->data = value;
	  newNode->next = NULL;
	
	  if (head == NULL) {
	  head = newNode;
	  return;
	  }
	
	  while (last->next != NULL) {
	  last = last->next;
	  }

	last->next = newNode; }

Deletion of a node	void deleteNode(char value) {
	Node *temp = head, *prev = NULL;
	
	if (temp != NULL && temp->data == value) {
	head = temp->next;
	delete temp;
	std::cout << "Node '" << value << "' deleted." << endl;
	return;
	}
	
	while (temp != NULL && temp->data != value) {
	prev = temp;
	temp = temp->next;
	}
	
	if (temp == NULL) {
	cout << "Node '" << value << "' not found." << endl;
	return;
	}
	
	prev->next = temp->next;
	delete temp;
	std::cout << "Node '" << value << "' deleted." << endl;
	}

Table 3-2. Code for the List Operations

a.	**Source Code**	std::cout << "a. Traversal of original list:" << endl; displayList(head);
	Console	a. Traversal of original list: List: CPE101

b.	**Source Code**	std::cout << "b. Insert 'G' at the start of the list:" << endl;
		insertAtHead('G');
		displayList(head);

	Console	b. Insert 'G' at the start of the list: List: GCPE101
c.	**Source Code**	std::cout << "c. Insert 'E' after the node containing 'P':" << endl;

		Node *prevNode = findNode('P');
		insertAfter(prevNode, 'E');
		displayList(head);

	Console	c. Insert 'E' after the node containing 'P': List: GCPEE101

d.	**Source Code**	std::cout << "d. Delete the node containing 'C':" << endl;
		deleteNode('C');
		displayList(head);

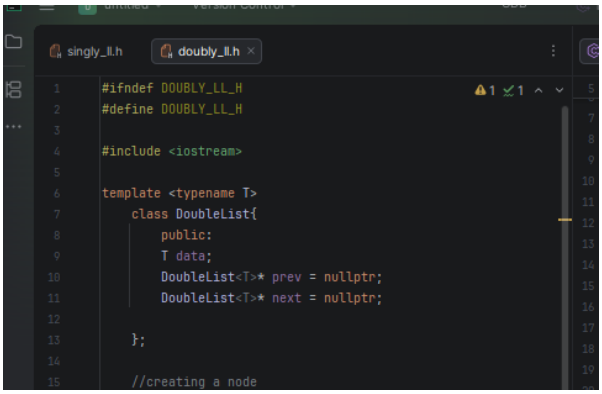
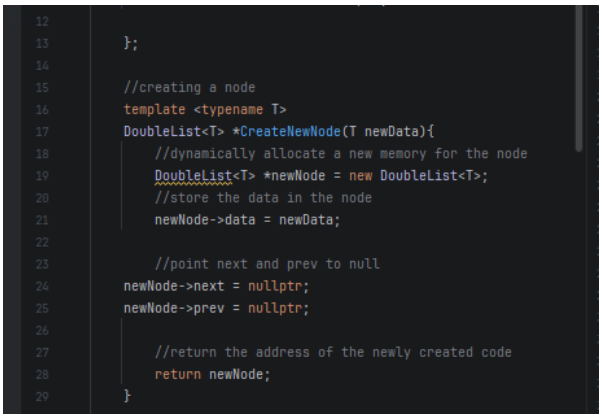
	Console	d. Delete the node containing 'C': List: GPEE101

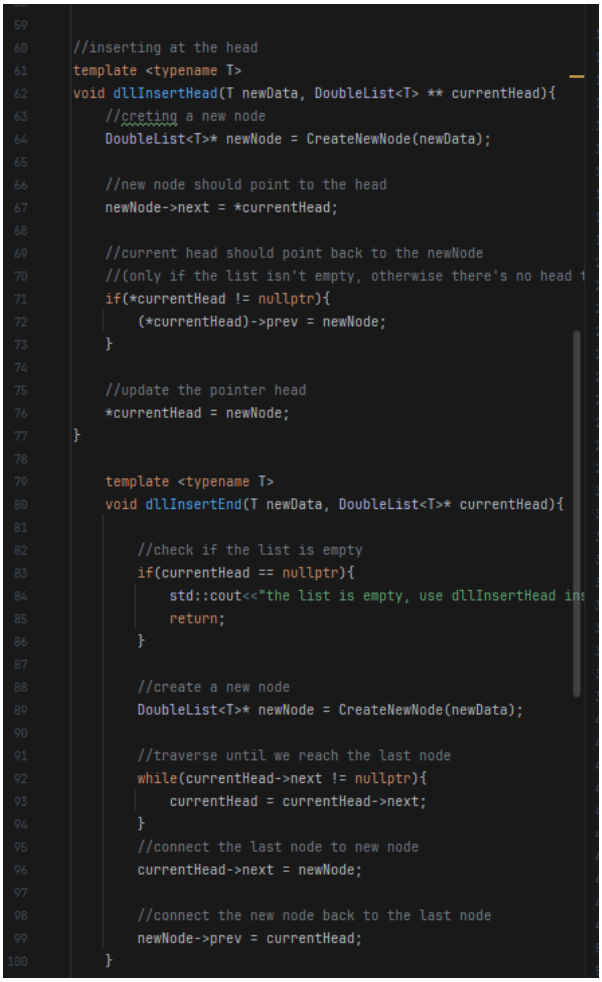
e.	**Source Code**	std::cout << "e. Delete the node containing 'P':" << endl;
		deleteNode('P');
		displayList(head);

	Console	e. Delete the node containing 'P': List: GEE101
f.	**Source Code**	std::cout << "f. Final list:" << endl; displayList(head);
	Console	f. Final list: List: GEE101

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis

	Defines the node structure: a data field and two pointers, next and prev, both defaulted to nullptr. The addition of prev is the structural basis for the entire doubly linked list.
	Allocates a new node, assigns the given value, initializes both pointers to nullptr, and returns the node. Used by all subsequent insertion functions.
	This checks for an empty list. Traverses forward via next, printing each value and storing the last visited node in tail. Then traverses backward from tail via prev, printing values in reverse.



```
//inserting at the head
template <typename T>
void dllInsertHead(T newData, DoubleList<T> ** currentHead){
    //creting a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    //new node should point to the head
    newNode->next = *currentHead;

    //current head should point back to the newNode
    //(only if the list isn't empty, otherwise there's no head t
    if(*currentHead != nullptr){
        (*currentHead)->prev = newNode;
    }

    //update the pointer head
    *currentHead = newNode;
}

    template <typename T>
    void dllInsertEnd(T newData, DoubleList<T>* currentHead){

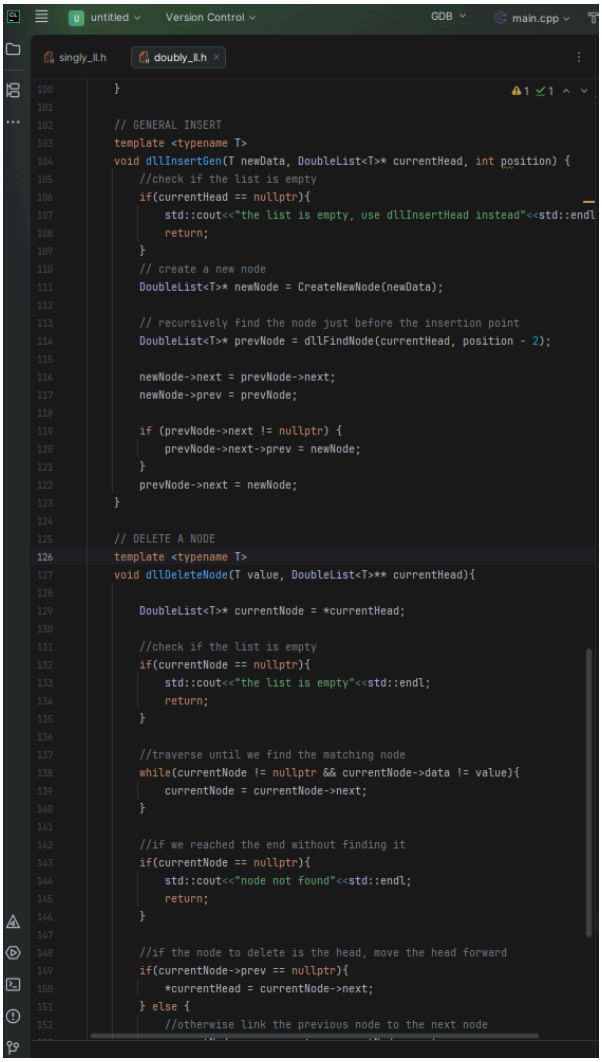
        //check if the list is empty
        if(currentHead == nullptr){
            std::cout<<"the list is empty, use dllInsertHead ins
            return;
        }

        //create a new node
        DoubleList<T>* newNode = CreateNewNode(newData);

        //traverse until we reach the last node
        while(currentHead->next != nullptr){
            currentHead = currentHead->next;
        }
        //connect the last node to new node
        currentHead->next = newNode;

        //connect the new node back to the last node
        newNode->prev = currentHead;
    }
```

The dllInsertHead creates a node, links it to the current head, updates the old head's prev if one exists, and reassigns the head pointer. dllInsertEnd rejects an empty list with a message, otherwise traverses to the last node and appends the new node, setting its prev accordingly.



The dllInsertGen locates the node preceding the target position via dllFindNode, then splices the new node in, updating next/prev links on both sides. Note: the result of dllFindNode is not checked for nullptr before use. dllDeleteNode searches for a matching node, reports if none is found, then relinks the surrounding nodes' next and prev pointers before freeing the deleted node.

Table 3-4. Modified Operations for Doubly Linked Lists

CODE (MAIN)

```
#include "circular_ll.h"
#include <iostream>
#include <string>

int main(){

   CircularList<std::string>* playlist = nullptr;
   CircularList<std::string>* current = nullptr;

   //songs for the playlist
   cllInsertEnd<std::string>("I really want to stay at your house", &playlist);
   cllInsertEnd<std::string>("TungTungTungSahu Remix", &playlist);
   cllInsertEnd<std::string>("COCOMELON - Daddy Finger", &playlist);
   current = playlist;

   int choice = 0;
   while(choice != 6){

       std::cout<<"\n==== SONY Song Playlist Menu ====\n";
```

```
        std::cout<<"1. Play all songs\n";
        std::cout<<"2. Next song\n";
        std::cout<<"3. Previous song\n";
        std::cout<<"4. Insert song\n";
        std::cout<<"5. Remove song\n";
        std::cout<<"6. Exit\n";
        std::cout<<"Enter choice: ";
        std::cin>>choice;
        std::cin.ignore();

        if(choice == 1){
            cllPlayAll(playlist);
        }
        else if(choice == 2){
            current = cllNext(current);
            if(current != nullptr){
                std::cout<<"Now on: "<<current->data<<std::endl;
            }
        }
        else if(choice == 3){
            current = cllPrevious(current);
            if(current != nullptr){
                std::cout<<"Now on: "<<current->data<<std::endl;
            }
        }
        else if(choice == 4){
            std::string songName;
            std::cout<<"Enter song name to add: ";
            std::getline(std::cin, songName);
            cllInsertEnd<std::string>(songName, &playlist);

            if(current == nullptr){
                current = playlist;
            }
        }
        else if(choice == 5){
            std::string songName;
            std::cout<<"Enter song name to remove: ";
            std::getline(std::cin, songName);
            cllDeleteNode<std::string>(songName, &playlist);

            current = playlist; //reset just to be safe
        }
        else if(choice == 6){
            std::cout<<"Exiting playlist.\n";
        }
        else{
            std::cout<<"invalid choice, try again\n";
        }
    }

    return 0;
}
```

CODE (HEADER)

```
#ifndef CIRCULAR_LL_H
#define CIRCULAR_LL_H

#include <iostream>
#include <string>

//node for a song
template <typename T>
class CircularList{
    public:
    T data;
    CircularList<T>* next = nullptr;
};

//add a song
template <typename T>
void cllInsertEnd(T newData, CircularList<T>** head){

    //make the new node
```

```
    CircularList<T>* newNode = new CircularList<T>;
    newNode->data = newData;

    //if playlist is empty, new node points to itself
    if(*head == nullptr){
        newNode->next = newNode;
        *head = newNode;
        return;
    }

    //find the last node by walking until we get back to head
    CircularList<T>* temp = *head;
    while(temp->next != *head){
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->next = *head;
}

//remove a song by its name?
template <typename T>
void cllDeleteNode(T findData, CircularList<T>** head){

    if(*head == nullptr){
        std::cout<<"the playlist is empty\n";
        return;
    }

    CircularList<T>* curr = *head;
    CircularList<T>* prev = nullptr;

    //just one song in the playlist
    if(curr->next == curr && curr->data == findData){
        delete curr;
        *head = nullptr;
        return;
    }

    bool found = false;
    do{
        if(curr->data == findData){
            found = true;
            break;
        }
        prev = curr;
        curr = curr->next;
    } while(curr != *head);

    if(!found){
        std::cout<<"song not found\n";
        return;
    }

    //if we're deleting the head, we need to fix the last node's next too
    if(curr == *head){
        //find the last node
        CircularList<T>* last = *head;
        while(last->next != *head){
            last = last->next;
        }
        *head = curr->next;
        last->next = *head;
    } else {
        prev->next = curr->next;
    }

    delete curr;
}

//play every song once around the loop
template <typename T>
void cllPlayAll(CircularList<T>* head){
    if(head == nullptr){
```

```
        std::cout<<"the playlist is empty\n";
        return;
    }

    CircularList<T>* temp = head;

    std::cout<<"Now playing: \n";
    do{
        std::cout<<temp->data<<" -> ";
        temp = temp->next;
    } while(temp != head);

    std::cout<<"(back to start)\n";
}

//go to the next song
template <typename T>
CircularList<T>* cllNext(CircularList<T>* current){
    if(current == nullptr){
        std::cout<<"the playlist is empty\n";
        return nullptr;
    }
    return current->next;
}

//goes to the previous song and tries to find the song right before the current one
template <typename T>
CircularList<T>* cllPrevious(CircularList<T>* current){
    if(current == nullptr){
        std::cout<<"the playlist is empty\n";
        return nullptr;
    }

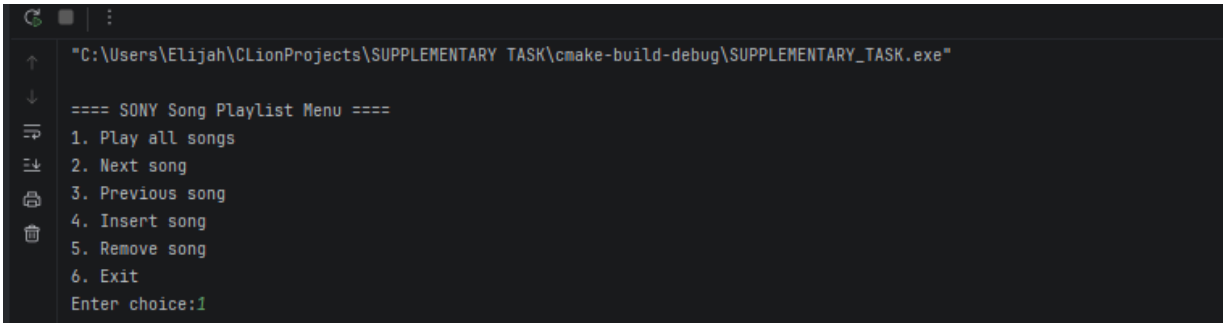
    if(current->next == current){
        return current;
    }

    CircularList<T>* temp = current;
    while(temp->next != current){
        temp = temp->next;
    }
    return temp;
}

#endif
```

OUTPUTS

Function 1



Function 2

```
6. Exit
Enter choice:2

Now on: TungTungTungSahu Remix

==== SONY Song Playlist Menu ====
1. Play all songs
2. Next song
3. Previous song
4. Insert song
5. Remove song
6. Exit
```

Function 3

```
==== SONY Song Playlist Menu ====
1. Play all songs
2. Next song
3. Previous song
4. Insert song
5. Remove song
6. Exit
Enter choice:3

Now on: I really want to stay at your house
```

Function 4

```
Now on: I really want to stay at your house

==== SONY Song Playlist Menu ====
1. Play all songs
2. Next song
3. Previous song
4. Insert song
5. Remove song
6. Exit
Enter choice:4

Enter song name to add:Gotoubun no Kimochi
```

Function 5

```


==== SONY Song Playlist Menu ====
1. Play all songs
2. Next song
3. Previous song
4. Insert song
5. Remove song
6. Exit
Enter choice:5

Enter song name to remove:COCOMELON - Daddy Finger


==== SONY Song Playlist Menu ====
1. Play all songs
2. Next song
3. Previous song
4. Insert song
5. Remove song
6. Exit
Enter choice:1

Now playing:
I really want to stay at your house -> TungTungTungSahu Remix -> Gotoubun no Kimochi -> (back to start)
```

Function 6



Conclusion

This activity provided more understanding of the limitations of standard singly and doubly linked lists when applied to systems requiring continuous traversals, such as a music playlist that loops back to its first song after the last one concludes. In working through this, the concept of a circular linked list was introduced, wherein the final node connects back to the head rather than pointing to nullptr. To add, particularly the adoption of a do...while loop in place of a standard while loop. Comparing a doubly linked circular list against a simplified singly linked version further clarified the trade-off introduced by the prev pointer, demonstrating that adding it enables immediate backward traversal, while its absence requires traversing nearly the entire list to achieve the same outcome.

In addition, the activity has shown the importance of reusability in code design and how single and double lists come to play. Overall, the topics covered in this activity included circular linked lists, the design trade-offs between singly and doubly linked structures, node insertion and deletion within a circular context, traversal and loop control techniques, and pointer manipulation using next and prev. It was difficult but thanks to help of some friends I was able to grasp some of the complex concepts when it comes to this topic.